

Claude Code Workflow Playbook

From Lab to Real Work - Your First Week (August 2026 edition)

The 5-Minute Setup (any project)

- 1. `cd` into the project
- 2. Run `/init` - Claude scans and drafts a `CLAUDE.md`; trim to policy (< 200 lines):

```
# Project: [Name]
## Tech Stack [language, framework, key deps]
## Architecture [where things live]
## Conventions [naming, errors, tests]
## Never [things Claude must never do here]
```

- 3. Verify: "What conventions does this project follow?"
- 4. Check `/memory` - auto memory will accumulate build/test facts on its own; you own the rules, Claude owns the learnings.

When to Use What

You want to...	Do this	Why
Quick fix, single file	Ask naturally (Default mode)	Low risk, fast feedback
Multi-file change	Shift+Tab -> Plan , approve, then execute	Plan mode can't write - safe to think
Well-scoped grunt work	Shift+Tab -> Auto (with permission rules set)	Autonomy inside your fence
Explore unfamiliar code	"Explain <code>@src/auth/</code> - what does this do?"	<code>@</code> scopes the reading
Repetitive task (3+ times)	<code>CLAUDE.md</code> rule or a skill	Don't repeat yourself to Claude either
Risky (DB, deploy, delete)	Plan mode + review every diff + deny rules	The fence, not vibes

Mode cycle (Shift+Tab): **Default -> AcceptEdits -> Plan -> Auto**. Auto mode ships with two-layer safety (injection probe + transcript classifier) and `rm -rf` guards - but your `deny/ask/allow` rules are the real control. Deny is evaluated first.

The Prompt Quality Ladder

- **L1 vague:** "Add authentication" - works, wastes iterations
- **L2 specific (~70% first-try):** "Add JWT auth to `@src/api/routes.js` using `jsonwebtoken`. Protect `/api/*` except `/api/health`."
- **L3 contextual (~90%):** L2 + "secret in `process.env.JWT_SECRET`, 24h expiry, follow `@src/middleware/errorHandler.js` patterns, tests like `@tests/api/`."

The rule: whatever you'd correct in review - write it into the prompt instead.

Checkpoints & Rewind - the Recovery Playbook

Claude Code checkpoints continuously. **Double-tap Esc** opens the rewind menu.

Symptom	Restore
Bad file changes	code
Derailed/poisoned conversation	conversation
Both	both
Damage predates checkpoints (teammate's merge)	Rebuild: describe what's wrong + @reference files + what done looks like

Rewind beats arguing with contaminated context - always.

The Self-Healing Loop - When to Intervene

LET IT RUN: import errors, type mismatches, missing config, readable test failures.

INTERRUPT (Esc): rewriting files you didn't ask about; same error 3+ times; wrong architecture; burning tokens on something you know.

After interrupting: give a correction, not "try again." Or double-Esc and rewind - cheaper than correcting.

Context Discipline

- Same task = keep context; new task = `/clear`
- `/compact` around 50% usage - don't wait for degradation
- `/context` to see what fills the window; `/usage` for plan limits and what consumed them; `/btw` for side questions that don't pollute
- `!command` runs shell and lets Claude react to the output - no copy-paste round trip
- Add Compact Instructions to CLAUDE.md so compaction preserves architecture decisions (see Templates handout)
- Multi-session work: end with "Write progress to HANDOFF.md - what you tried, what worked, what didn't" and start the next session from that file

Session Cost Sense

Use Sonnet 5 for normal implementation and Opus 5 for complex reasoning when available. Identify the meter before estimating cost: subscription seats use shared account allowances; API/cloud runs are token billed. Bound headless work with `--max-turns`, narrow tools, budgets, and telemetry.

Your First Week

Day	Do
1	<code>/init</code> + curated CLAUDE.md in ONE real repo; set deny/ask/allow via <code>/permissions</code>
2	First real ticket end-to-end: Plan mode -> execute -> <code>/review</code> -> PR
3	Write ONE skill for your most repeated review ask; eval it (5+5 prompts)
4	Wire the PostToolUse format hook + PreToolUse secret blocker from Lab 2B
5	Run <code>scripts/ci-review.sh</code> style headless check on a branch; demo to the team; propose the plugin

Weekly Hygiene

- `/doctor` - auto-fix issues; prune flagged unused skills/MCP servers; fix slow hooks
- Rerun skill evals after model updates
- Promote recurring auto-memory facts into `CLAUDE.md`; delete rules the model now does by default
- Check `/usage` trends; drop a model tier where quality allows

Which Extension, and When to Add It

Don't design your setup up front. Each row is a feeling you get while working - answer it then.

The trigger you will actually notice	Add
Claude gets a convention or command wrong twice	A line in <code>CLAUDE.md</code>
You type the same prompt to start a task, again	A user-invocable skill
You paste the same playbook into chat a third time	A skill
You keep copying data from a tab Claude cannot see	An MCP server
Claude reads many files just to find where a symbol lives	A code-intelligence plugin for your language
A side task floods your session with output you will not re-read	A subagent
You want it to happen every time, without being asked	A hook
A second repository needs the same setup	A plugin

*The same triggers tell you when to **update** what you already have. A repeated mistake is a `CLAUDE.md` edit, not a one-off correction in chat. A workflow you keep hand-tweaking is a skill that needs another revision.*

A rule is a request; a hook is a guarantee. "Never edit `.env`" in `CLAUDE.md` is followed almost always.

A `PreToolUse` hook that blocks the edit is followed *always*. If it must hold every time, make it a hook.

What Each Extension Costs You in Context

Feature	When it loads	Context cost
<code>CLAUDE.md</code>	Session start	Every request - keep it under 200 lines
Skills	Descriptions at start, body when used	Low. <code>disable-model-invocation: true</code> makes it zero until you invoke it
MCP servers	Tool <i>names</i> at start, schemas on demand	Low until a tool is actually used
Code intelligence	After edits, on symbol lookup	Low - often <i>replaces</i> broad file reads
Subagents	When spawned	Isolated - does not touch your main window
Hooks	On their event	Zero , unless the hook returns output

When the Same Thing Exists at Two Levels

Feature	How it resolves
CLAUDE.md files	Additive - every level contributes; more specific usually wins on conflict
Skills	Override by name: managed > user > project. Plugin skills are namespaced
Subagents	Override by name: managed > CLI flag > project > user > plugin
MCP servers	Override by name: local > project > user
Hooks	Merge - every registered hook fires for its event, whatever the source

The Three-Layer Stack

- 1. **Policy** - CLAUDE.md + permission rules (deny -> ask -> allow)
 - 2. **Automation** - skills, hooks, commands, plugins
 - 3. **Verification** - graders, evals, /review, headless CI checks
- Any single layer has gaps. All three together is an engineering system.
- (c) 2026 AIA Copilot - Claude Code Training